



Sistemas Operativos

Procesos Padres e hijos

EN LINUX



Procesos Padre/Hijo con FORK

En lenguaje C, un proceso hijo creado con `fork()` es un nuevo proceso que se crea a partir de un proceso padre existente. El proceso hijo es una copia exacta del proceso padre, con la única diferencia de que tiene su propio PID (identificador de proceso).

Cuando se llama a `fork()` en un proceso padre, se crea un proceso hijo y ambos procesos comienzan a ejecutar el mismo código a partir de la llamada a `fork()`. El proceso padre y el proceso hijo comparten el mismo espacio de memoria “virtual”, que se divide en dos partes: una parte para el proceso padre y otra para el proceso hijo. Cada proceso tiene su propia copia de las variables y los datos del programa, pero ambos procesos comparten el mismo código del programa.

Después de crear el proceso hijo, el proceso padre puede continuar ejecutando su propio código, mientras que el proceso hijo también puede ejecutar su propio código. Debido a que ambos procesos comparten el mismo código, es posible que se realicen operaciones simultáneamente en ambas copias de las variables y los datos del programa.

Es importante tener en cuenta que el proceso hijo tiene su propia copia de las variables y los datos del programa, por lo que cualquier cambio realizado en el proceso hijo no se reflejará en el proceso padre y viceversa. Además, cada proceso tiene su propia identidad y se ejecuta de forma independiente del otro proceso. El proceso hijo puede continuar ejecutándose incluso después de que el proceso padre haya terminado, y viceversa.



Ejemplo simple del uso de Fork():

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

int main() {

    pid_t pid;

    // Crear un nuevo proceso hijo

    pid = fork();

    // Verificar si se ha creado el proceso hijo

    if (pid < 0) {

        fprintf(stderr, "Error al crear el proceso hijo\n");

        exit(1);

    } else if (pid == 0) {

        // Este es el proceso hijo

        printf("Hola, soy el proceso hijo (PID %d)\n", getpid());

        exit(0);

    } else {

        // Este es el proceso padre

        printf("Hola, soy el proceso padre (PID %d)\n", getpid());

        printf("El proceso hijo tiene el PID %d\n", pid);

    }

    return 0;

}
```

En este ejemplo, `fork()` se usa para crear un nuevo proceso hijo. Después de llamar a `fork()`, el proceso padre y el proceso hijo ejecutan el mismo código a partir de la llamada a `fork()`. La única diferencia es que el valor de retorno de `fork()` es diferente en cada proceso. En el proceso padre, el valor de retorno de `fork()` es el PID (identificador de proceso) del proceso hijo, mientras que en el proceso hijo, el valor de retorno de `fork()` es 0.

El proceso hijo imprime un mensaje indicando que es el proceso hijo y luego sale del programa con `exit(0)`. El proceso padre imprime un mensaje indicando que es el proceso padre y luego imprime el PID del proceso hijo.



El uso de `fork()` en este ejemplo es simple, pero es importante tener en cuenta que `fork()` crea un proceso completamente nuevo que ejecuta el mismo código que el proceso padre. Por lo tanto, cualquier cambio en las variables o el estado del proceso padre no se reflejará en el proceso hijo, y viceversa. Además, es importante que el proceso padre espere a que el proceso hijo termine antes de salir del programa, de lo contrario, el proceso hijo podría quedar (huérfano) en ejecución incluso después de que el proceso padre haya terminado.

Un proceso hijo se convierte en huérfano cuando el proceso padre finaliza antes de que el proceso hijo termine de ejecutarse. Esto ocurre cuando el proceso padre sale del programa antes de que el proceso hijo termine de ejecutarse o si el proceso padre se interrumpe de alguna otra forma, como mediante una "señal."

Cuando un proceso hijo se convierte en huérfano, el proceso hijo todavía continúa ejecutándose en segundo plano, pero ya no tiene un padre en el árbol de procesos. En lugar de eso, el proceso hijo es adoptado por el proceso `init` del sistema operativo, que es el primer proceso que se ejecuta cuando se inicia el sistema operativo. El proceso `init` actúa como un "padre sustituto" para los procesos huérfanos.

Es importante tener en cuenta que cuando un proceso hijo se convierte en huérfano, puede haber algunas consecuencias no deseadas. Por ejemplo, si el proceso hijo crea otros procesos secundarios, es posible que estos procesos no se finalicen correctamente cuando el proceso hijo termina de ejecutarse. Además, si el proceso hijo utiliza recursos del sistema, como archivos o sockets, estos recursos pueden quedar en uso incluso después de que el proceso hijo termine de ejecutarse.

En general, se recomienda que los procesos hijos sean "recolectados" correctamente por el proceso padre para evitar que queden huérfanos. Esto se puede hacer utilizando la función `wait()` o `waitpid()` en el proceso padre para esperar a que el proceso hijo termine de ejecutarse antes de salir del programa.

La función `getpid()` se utiliza para obtener el identificador de proceso (PID) del proceso actual. El PID es un número único que se utiliza para identificar un proceso en el sistema operativo.

La función `getpid()` se encuentra en la biblioteca estándar de C `unistd.h`. Su sintaxis es la siguiente:

```
#include <unistd.h>
```

```
pid_t getpid(void);
```

La función `getpid()` no tiene argumentos y devuelve el PID del proceso actual como un valor entero de tipo `pid_t`. El valor devuelto se puede utilizar para identificar el proceso en el sistema operativo y para realizar operaciones relacionadas con el proceso, como enviar "señales" al proceso o esperar a que el proceso termine de ejecutarse.



Un ejemplo de uso de la función `getpid()` en C sería:

```
#include <stdio.h>

#include <unistd.h>

int main() {

    pid_t pid;

    pid = getpid();

    printf("El PID del proceso actual es %d\n", pid);

    return 0;

}
```

Este programa utiliza la función `getpid()` para obtener el PID del proceso actual y lo muestra en la pantalla utilizando la función `printf()`.